FINAL COURSE PROJECT REPORT

Credence AI

AI Based Digital Certificate Tamper Detection and Verification System

Artificial Intelligence Course Project

CSE311

Submitted by

Pranathi TR - 2024BCS0069

Rupasri M - 2024BCS0077

Peswika Bavagni - 2024BCS0257

Challa vidya - 2024BCS0157

Semester 5, Batch 2

Submitted to

Prajeesh Sir

Indian Institute of Information Technology Kottayam

October 2026

https://github.com/ptr25/credence-ai

Abstract

Digital certificates can undergo harmless changes such as compression and PDF re-exporting, as well as deliberate alterations to credential content. Cryptographic hashing detects byte-level changes but does not distinguish these two cases. Credence AI investigates reference-based visual tamper detection using transfer learning and supervised classification. An ImageNet-pretrained ResNet18 extracts one global and nine regional visual embeddings from each normalized certificate image. Cosine similarities are combined with structural similarity and mean absolute pixel difference to form a five-dimensional comparison vector. A Random Forest classifier is trained on synthetic reference-candidate pairs and selected against a Logistic Regression baseline using validation F1. The dataset contains 540 pairs generated from 36 base certificates, with all variants of a base assigned to the same training, validation or test partition. On 90 held-out synthetic pairs, the selected model achieved 100 percent accuracy, precision and recall, with F1 and ROC-AUC equal to 1.00. Global-only analysis achieved 81.11 percent accuracy. A Next.js interface and FastAPI backend demonstrate issuance, visual verification, difference maps and revocation. SHA-256, QR links and an LSB identity prototype provide supporting integrity and identification signals. These results demonstrate the controlled prototype; they do not establish performance on independently collected certificates, unfamiliar layouts or realistic adversarial edits.

Keywords: certificate tamper detection; transfer learning; ResNet18; patch embeddings; Random Forest; structural similarity.

Scope of the report

This report describes the implemented system, its synthetic dataset, learning pipeline, application workflows and measured evaluation. The trained component is the five-feature classifier. ResNet18 is reused with fixed pretrained weights and is not fine-tuned. Verification is relative to a trusted registered reference; issuer legitimacy is not inferred from an image alone.

Abbreviations

Term	Meaning
CNN	Convolutional neural network
SSIM	Structural Similarity Index
RF	Random Forest
ROC AUC	Area under the receiver operating characteristic curve
LSB	Least significant bit
API	Application programming interface

Contents

Abstract 2

1 Introduction and background 4

2 System architecture 5

3 Application and data design 6

4 Preprocessing and neural feature extraction 7

5 Regional and structural comparison 8

6 Synthetic dataset construction 9

7 Classifier training and experimental protocol 10

8 Evaluation results 11

9 Feature ablation and interpretation 12

10 Issuance and verification workflows 13

11 Demonstration case study 14

12 User interface 15

13 Testing and limitations 16

14 Conclusion and reproducibility 17

References and development acknowledgement 18

15 Key code and repository 19

Figures

Figure 1 System architecture 5

Figure 2 Held out confusion matrix 11

Figure 3 ROC curve 12

Figure 4 Grade alteration difference map 14

Figure 5 Institution dashboard 15

Numbered references identify external research and official library documentation. Implementation and result claims are supported by the project source files, saved dataset metadata, trained model, evaluation metrics and verification records.

1 Introduction and background

1 1 Problem statement

Academic certificates contain identity, course and performance information whose alteration can affect the trust placed in a credential. An exact-file hash can detect any byte modification, but compression or image-based PDF re-export can also change bytes without changing the visible qualification. A useful verifier therefore needs evidence about both file integrity and visual content.

Credence AI addresses a reference-based problem: given a registered authentic certificate and a submitted candidate, classify the candidate as visually authentic or harmlessly transformed, or as suspected content tampering. Registration and issuer trust are explicit assumptions. The system is not a universal detector of forged credentials with no known reference.

1 2 Objectives

- Extract repeatable neural visual fingerprints from registered and submitted certificates.
- Improve sensitivity to local changes through a 3 by 3 patch comparison.
- Combine learned visual similarity with structural and pixel-level measurements.
- Train a supervised classifier on labeled comparison pairs and evaluate it on grouped held-out data.
- Demonstrate the complete issuance, verification and revocation workflow through a usable web application.

1 3 Research foundations

Residual networks introduced by He et al. use shortcut connections to support the training of deep image representations [1]. Credence AI adopts a pretrained ResNet18 as a fixed feature extractor. This choice makes a small-data course prototype feasible without claiming certificate-specific CNN training.

Random Forests combine multiple randomized decision trees [2]. Their ability to learn nonlinear feature interactions motivates their use for five visual comparison measurements. Logistic Regression provides a simpler baseline. SSIM evaluates structural image similarity using local image statistics [3]; in this system it complements neural comparison rather than replacing the classifier.

1 4 Practical contribution

The contribution is an integrated, reproducible prototype combining frozen CNN representations, localized comparisons, a trained binary classifier and a registry-backed application. QR links, hashing and a hidden-ID prototype support the demonstration, while measurable classification results remain the central AI component.

2 System architecture

The application has a browser interface, one Python backend and a local registry. This architecture keeps certificate generation, model inference and audit data in a single deployable prototype. The browser communicates with the backend over HTTP and does not execute the neural network.

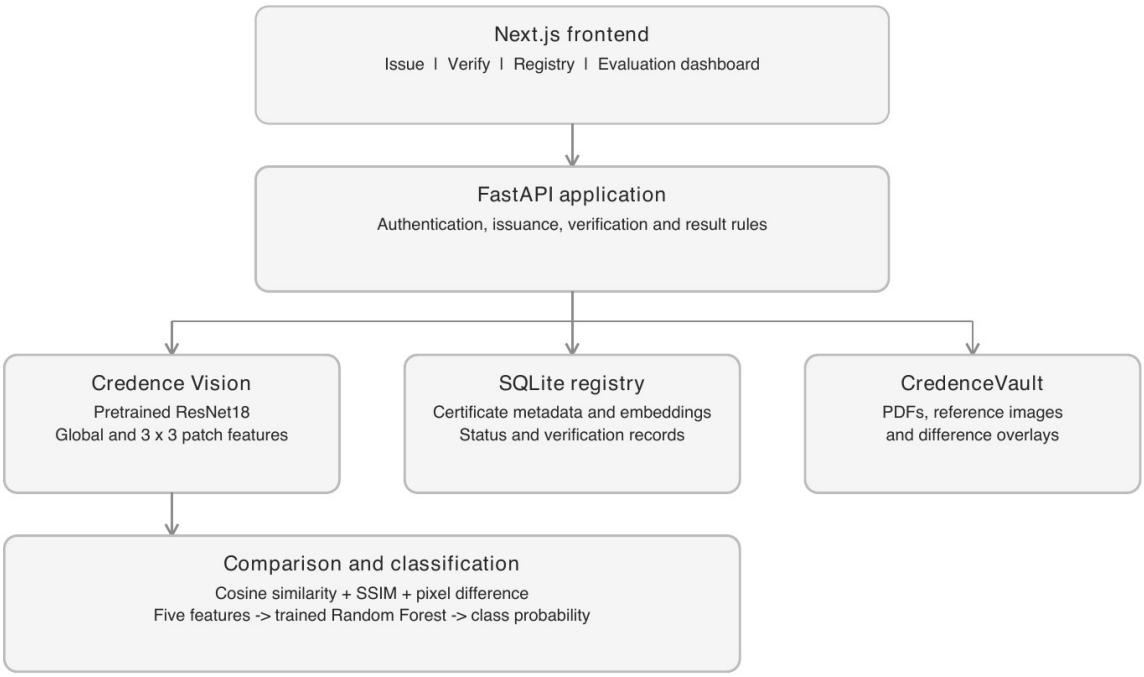


Figure 1 Application architecture and the visual classification subsystem

Layer	Implemented technology	Responsibility
Presentation	Next.js React TypeScript	Forms, upload, registry, results and evaluation views
Backend	Python FastAPI	Authentication, issuance and verification APIs
AI inference	PyTorch torchvision	Cached pretrained ResNet18 embeddings
Classification	scikit-learn joblib	Classifier training, inference and model persistence
Image and PDF	PyMuPDF Pillow ReportLab	Rendering, transformations and certificate generation
Persistence	SQLite and local files	Records, embeddings, PDFs and difference maps

The frontend runs at localhost port 3000 and the API at port 8000. JSON is used for form data and results; document uploads use multipart form data. Python dependencies and frontend dependencies are recorded separately. The project uses local storage rather than distributed infrastructure.

3 Application and data design

3 1 Registry and storage

SQLite contains users, certificates and verifications. Certificate rows include an auto-increment record key, unique certificate ID, status, timestamp and a JSON data field. That JSON stores student and course details, document hash, global and patch embeddings, reference-image path and PDF path. Verification rows store the certificate ID, timestamp and JSON analysis values. Embeddings are serialized arrays rather than a separate vector database.

CredenceVault is the project name for local filesystem storage. Final PDFs, normalized reference images, demonstration copies and difference overlays are saved there. The registry stores paths to these assets. A write transaction and SQLite ID allocation prevent two issuances from selecting the same counter.

3 2 API surface

Method	Route group	Purpose
POST	/api/auth/login	Return a temporary bearer session token
POST	/api/certificates/issue	Generate and register a certificate
GET	/api/certificates and /{id}	List or retrieve certificate information
GET	/api/certificates/{id}/download	Download the registered PDF
POST	/api/certificates/{id}/revoke	Revoke with a mandatory reason
POST	/api/verify	Analyze a PDF or image upload
GET	/api/verifications and /api/analytics	Read history and aggregate activity
GET	/api/ai/status and /api/ai/metrics	Read engine details and evaluation
POST	/api/demo/{id}/transform or /tamper	Create a real variant and run verification

3 3 Authentication and client behavior

The demonstration account uses PBKDF2-HMAC-SHA256 with 200000 iterations. A successful login creates a random token valid for eight hours. The frontend stores the token locally and sends it in an Authorization bearer header. Server-side sessions are in memory, so a backend restart invalidates them. These tokens are not JWTs.

Administrative operations require a session token. The public verifier accepts a supported document and an optional certificate ID. React state controls loading, errors and analysis display. The backend returns numerical model outputs; the frontend formats rather than invents those values. Public file routes and the fixed demo account require stronger controls before any production use.

4 Preprocessing and neural feature extraction

4 1 Shared input pipeline

A single-page PDF is rendered using PyMuPDF at matrix 1.0. Image files are converted to RGB. Portrait inputs are rotated by 90 degrees, and the image is resized to a 900 by 636 canvas using Lanczos resampling. The same normalization function is used for registration and verification. Multi-page PDFs are rejected and uploaded files are limited to 15 MB.

Each image supplied to ResNet18 then receives the pretrained weights transform: resize the shorter side to 256, center crop to 224 by 224, convert to a tensor and normalize RGB channels [4]. The channel means are 0.485, 0.456 and 0.406; standard deviations are 0.229, 0.224 and 0.225. These settings match the pretrained model input convention.

4 2 Fixed pretrained representation

The code loads resnet18 with ResNet18_Weights.DEFAULT, which selects ImageNet pretrained weights. The original final classification layer is replaced by torch.nn.Identity. The resulting output is the 512-dimensional representation before the final classifier. ImageNet pretraining is reused; no backpropagation or certificate-specific fine-tuning is performed on the CNN.

One full-page image and nine patches are stacked into a single batch. The CNN input has shape 10 by 3 by 224 by 224 and the embedding output has shape 10 by 512. The model is cached once, switched to evaluation mode and run inside torch.inference_mode. This avoids repeated initialization and gradient tracking during requests.

4 3 Embedding normalization and similarity

Each embedding is divided by its L2 norm, with a small lower bound to avoid division by zero. For a vector z, the normalized representation is z divided by the square root of the sum of its squared components. This reduces sensitivity to vector magnitude.

```
cosine(a, b) = (a dot b) / (norm(a) * norm(b))
```

For unit-length embeddings, cosine similarity becomes a dot product. The implementation clips reported similarities to the interval zero to one. These values measure closeness of learned representations; they are not percentages of unchanged text or probabilities of issuer legitimacy.

4 4 Preprocessing limitations

Orientation handling is a fixed landscape rotation rather than text-aware orientation estimation. No skew, perspective or translation registration is implemented. Center cropping can omit image edges, including patch edges. These choices are suitable for the controlled generated template but restrict robustness to arbitrary scans and layouts.

5 Regional and structural comparison

5 1 Patch analysis

The normalized canvas is divided into nine corresponding 300 by 212 regions. Local embeddings preserve more region-specific information than one pooled page representation. The verifier compares corresponding reference and candidate patches and records all nine cosine similarities, their mean, their minimum and the index of the minimum. Patch indices run row by row from one to nine.

A small grade alteration can have a limited effect on the global vector because most of the document is unchanged. Regional features give the classifier additional evidence. The most anomalous patch is a relative minimum, not independently a tampering decision; even identical inputs have a minimum subject to floating-point rounding.

5 2 Structural and pixel measurements

SSIM compares luminance, contrast and structural information [3]. The implementation applies the scikit-image function to normalized RGB arrays with channel_axis equal to 2 and data_range equal to 1. Negative aggregate SSIM values are clamped to zero. SSIM is a deterministic image-processing measurement, not an additional trained neural network.

Mean absolute pixel difference averages the absolute reference-candidate difference over all RGB channels after values are scaled to zero through one. The difference image is amplified and blended into red over the candidate. It visualizes changed pixels rather than model attention or feature attribution.

5 3 Final comparison vector

Feature	Definition	Expected interpretation
Global similarity	Cosine of full-page embeddings	Overall learned visual similarity
Average patch similarity	Mean of nine regional cosines	Typical local similarity
Minimum patch similarity	Smallest regional cosine	Least similar region
Structural similarity	SSIM over normalized RGB pages	Structural image consistency
Mean pixel difference	Average absolute RGB difference	Magnitude of pixel-level change

The classifier receives these five values in this order. Neither the file hash nor the watermark status is used as a learned feature. The application combines those supporting signals with the classifier prediction when producing its final result. The forest can learn feature relationships, so no single similarity value should be treated as a universal authenticity threshold.

6 Synthetic dataset construction

The dataset is generated within the project and contains no independently collected institutional certificate corpus. Thirty-six base certificates share one controlled layout. Their names, student IDs, courses, grades and certificate IDs vary. Four name patterns and four course names are cycled, names receive numeric suffixes, and three grade values are used.

6 1 Labeled transformations

Class	Variant	Generation setting
Authentic	Original	Unchanged reference image
Authentic	JPEG compression	Quality 75
Authentic	Resize	720 by 509 and back to the normalized size
Authentic	PDF re-render	Image PDF export and rendering
Authentic	Blur	Gaussian radius 0.35
Authentic	Brightness and contrast	Separate variants using factors 0.96 and 0.95
Authentic	Noise	Gaussian pixel noise with standard deviation 1
Tampered	Grade alteration	Cover the grade region and write Grade O
Tampered	Name course and ID edits	Three separate controlled text-region replacements
Tampered	Cover insert and remove	Three separate region modifications

There are eight authentic or transformed variants and seven tampered variants per base. Thus 36 times 15 produces 540 reference-candidate pairs: 288 authentic or transformed and 252 tampered. A pair becomes one five-feature training example. This count is not 540 independently sourced certificates.

6 2 Reproducibility and realism

A fixed random seed of 42 controls stochastic generation. The dataset JSON records the base identifier, variant type, class label and extracted feature vector. Candidate images are generated programmatically rather than all being retained as independent files.

Tampered examples use known rectangular coordinates. The grade variant writes Grade O, while other text edits insert a common replacement string into their designated region. These alterations do not preserve the original typography and are not located by OCR. The experiment therefore measures recognition of controlled visual edits. Some harmless variants may also preserve nearly identical pixels, which makes parts of the dataset easier.

7 Classifier training and experimental protocol

7 1 Grouped partitioning

Base certificate identifiers are shuffled with seed 42. All variants of each base are assigned together, giving 24 training bases, six validation bases and six test bases. This prevents variants of an identical reference from being distributed across the three partitions.

Partition	Bases	Pairs	Authentic	Tampered
Training	24	360	192	168
Validation	6	90	48	42
Test	6	90	48	42
Total	36	540	288	252

Training fits the classifiers, validation selects the model and the held-out test partition supplies the reported evaluation. Grouping reduces near-duplicate leakage but does not remove shared-template or shared-transformation similarity across partitions. No cross-validation or independent external dataset is used.

7 2 Candidate models

Logistic Regression is trained in a pipeline with StandardScaler and a maximum of 1000 optimization iterations. Random Forest uses 150 trees, maximum depth seven, balanced class weights and seed 42. Both receive the same five-feature vectors. The classifier label is zero for authentic or transformed pairs and one for tampered pairs.

Candidate	Validation F1	Selection
Standardized Logistic Regression	0.9231	Baseline
Random Forest	1.0000	Selected

The model with the higher validation F1 is saved with joblib. The selected Random Forest remains fitted on the training partition; the test partition is not used to select the model. New issuance stores reference embeddings and does not retrain either the CNN or the classifier.

7 3 Confidence interpretation

The forest supplies class probabilities by averaging the constituent trees probability estimates [5]. The reported confidence is the probability assigned to the predicted class. It is not a calibrated guarantee of correctness. The application sends predictions below 0.65 confidence for review; this is a manually chosen operational rule rather than the learned tampering boundary.

8 Evaluation results

The positive class is tampered. The selected model classifies all 90 held-out synthetic test pairs correctly: 48 authentic or transformed examples and 42 tampered examples. The counts are consistent with the saved metrics and the retained evaluation figure.

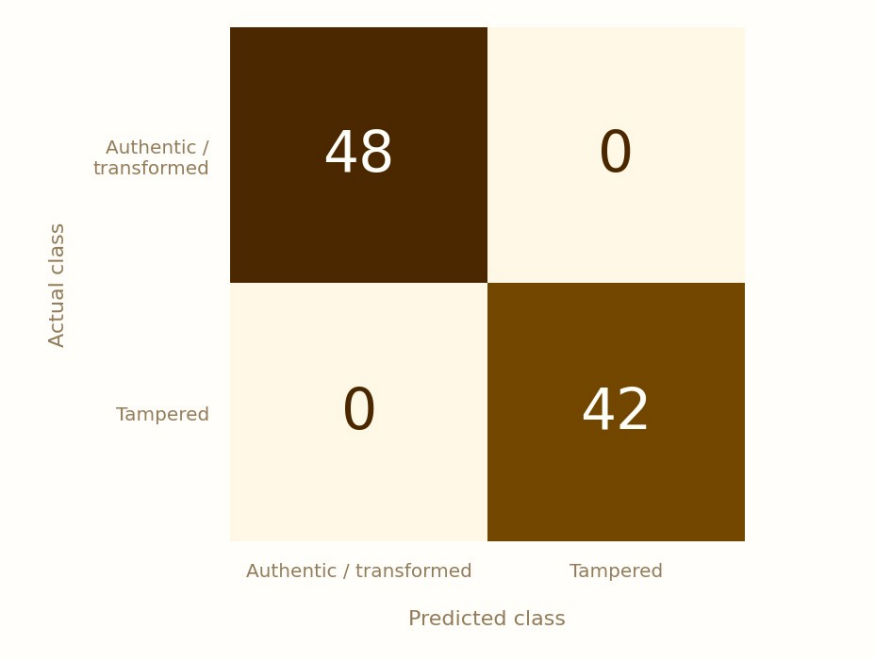


Figure 2 Confusion matrix on 90 test pairs with actual classes in rows

Measure	Calculation	Result
Accuracy	(TP + TN) / all pairs = 90 / 90	100.00 percent
Precision	TP / (TP + FP) = 42 / 42	100.00 percent
Recall	TP / (TP + FN) = 42 / 42	100.00 percent
F1 score	2PR / (P + R)	1.00
ROC AUC	Area under the ROC curve	1.00

The confusion matrix contains 48 true negatives, zero false positives, zero false negatives and 42 true positives. A false positive would flag a harmless example; a false negative would accept a tampered example. In this application, missed tampering is particularly consequential, although false alerts also reduce usability.

These results are bounded by six held-out base certificates and recurring generation patterns. They do not establish a zero error rate on real-world scans, unknown templates or adversarial alterations. The apparent perfect separation should be interpreted as success on a small controlled test, not universal reliability.

9 Feature ablation and interpretation

Three Random Forest models were evaluated with different feature subsets using the same grouped partition. The global-only baseline receives one similarity value; the regional model receives global, mean-patch and minimum-patch similarity; the full model adds SSIM and mean pixel difference.

Feature subset	Accuracy %	Precision %	Recall %	F1 %
Global only	81.11	82.05	76.19	79.01
Global and patches	100.00	100.00	100.00	100.00
Global patches SSIM and pixels	100.00	100.00	100.00	100.00

Adding regional features improves accuracy from 81.11 to 100 percent on this test partition. Both the regional and full feature sets reach the same score, so the experiment does not demonstrate a further accuracy improvement due to SSIM. Structural measurements remain complementary signals, but their incremental value needs a harder dataset.

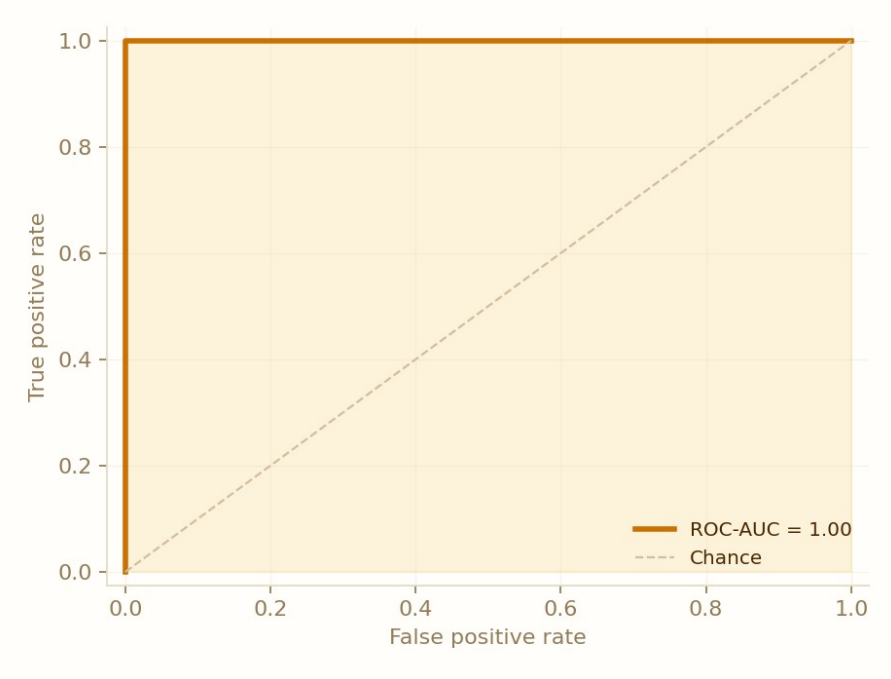


Figure 3 ROC curve for the selected classifier on the held out synthetic test set

ROC-AUC summarizes ranking as the threshold on the tampered-class score varies. It is a dataset-level metric, distinct from the confidence shown for one uploaded file. The ROC result supports separation within this test set; it does not compensate for limited dataset diversity.

10 Issuance and verification workflows

10 1 Certificate issuance

An authenticated administrator supplies student and credential details. The backend allocates a certificate ID, renders the ReportLab certificate with a QR link, embeds a CredenceID payload into image least-significant bits and places that image into the final PDF. SHA-256 is computed after finalization. The final document is rendered through the shared preprocessing function, its embeddings are extracted, and the reference assets and registry record are saved.

10 2 Document verification

The verifier accepts a document and optional ID, renders the input and attempts hidden-ID extraction. An entered ID takes precedence over the extracted ID. Once the registered reference is located, candidate embeddings and comparison features are calculated. Random Forest returns a class and probability; the backend compares SHA-256 values, checks registry status and saves the analysis with its visual difference overlay.

Application result	Decision rule
VERIFIED	Authentic or transformed prediction and matching hash
TRANSFORMED_COPY	Authentic or transformed prediction and different hash
TAMPERING_SUSPECTED	Tampered prediction
REVIEW_RECOMMENDED	Predicted-class probability below 0.65
REVOKED	Registry revocation overrides the visual result
NOT_FOUND	No registered reference is located

10 3 Supporting integrity signals

SHA-256 establishes exact-byte consistency with the registered file. It is not a signature. CredenceID stores the ID plus an eight-hex-character checksum in pixel LSBs; the unkeyed checksum can be recreated and may be destroyed by compression. Its status is displayed but does not independently override the verdict.

The QR carries a portal link with the certificate ID. It does not itself verify the file, and its localhost address is intended for the local demonstration. Revocation updates registry status without modifying the PDF. An unchanged revoked file can therefore remain visually consistent while the final result is REVOKED.

11 Demonstration case study

A retained demonstration uses Aarav Sharma certificate CRED-IIITK-2026-000003. An original PDF, a JPEG-transformed PDF and a grade-edited PDF were processed through the same verification function. The values below are taken from the saved application records; this illustration is separate from the held-out test metrics.

Signal	Original	Transformed	Tampered
Global similarity %	100.00	99.93	99.86
Mean patch similarity %	100.00	99.38	99.96
Minimum patch similarity %	100.00	98.39	99.67
SSIM %	100.00	98.80	99.45
SHA 256	Match	Different	Different
Model class	Authentic	Authentic	Tampered
Application result	Verified	Transformed copy	Tampering suspected

The tampered example can retain higher regional and structural similarity than the compressed example because its edit is localized. This supports interpreting multiple measurements together. The grade alteration produces the lowest neural patch similarity in region five. Its hidden ID remains recoverable, demonstrating that a valid watermark is not sufficient to establish unchanged content.



Figure 4 Actual pixel difference overlay for the grade altered demonstration copy

The red overlay is computed from absolute pixel differences and is not a neural attention visualization. Demonstration variants use the same controlled edit strategy as dataset construction, so the workflow demonstrates functional behavior rather than independent adversarial robustness.

12 User interface

The responsive interface separates institution tasks from the public verification flow. The dashboard displays registry counts and recent activity; issuance supplies a live layout preview; the registry provides search, status filters, download and administrative actions. Credence Vision displays measured evaluation results, and the public verifier presents actual model scores and difference maps.

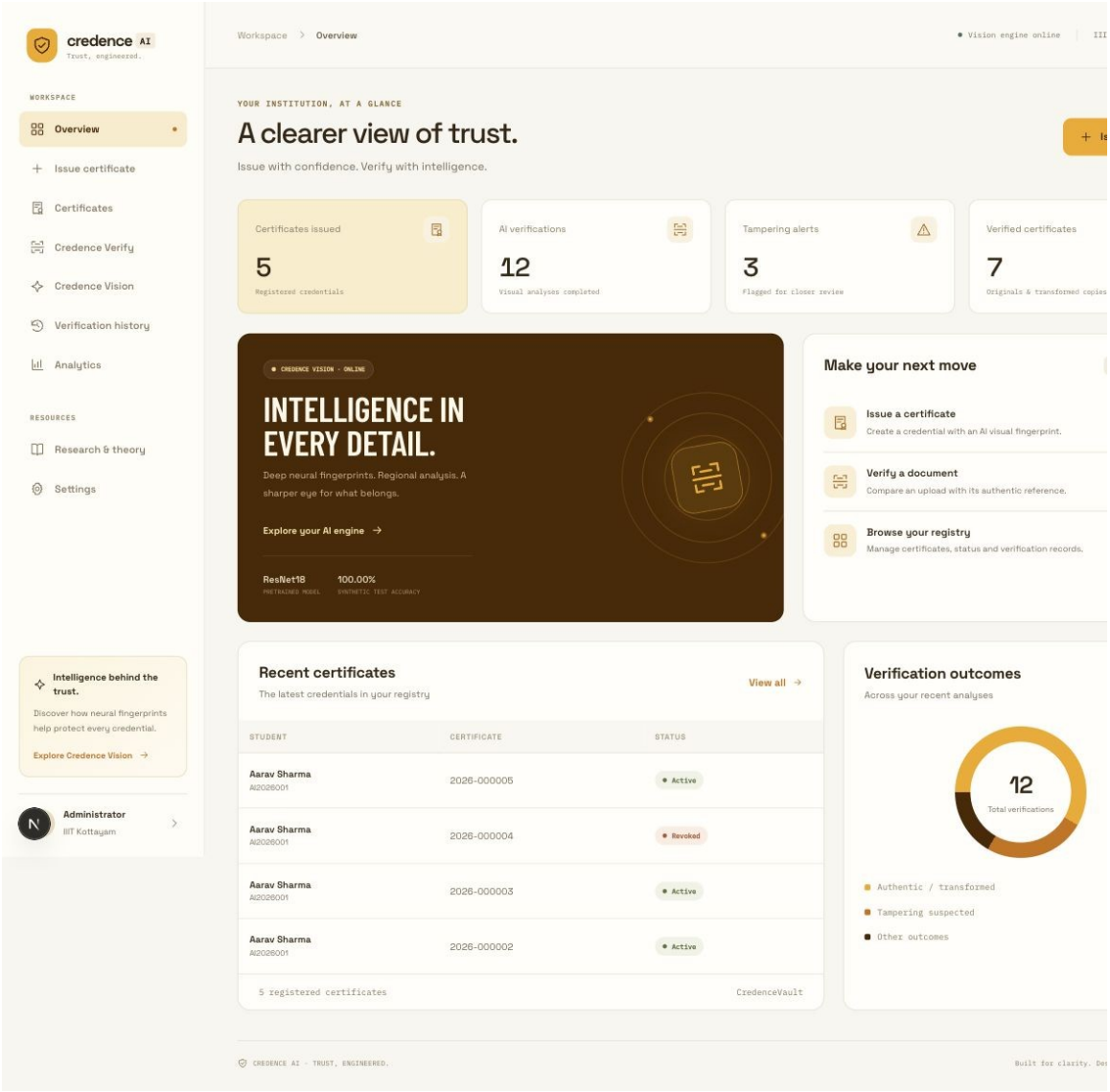


Figure 5 Implemented institution dashboard with live registry and verification counts

The visual design uses cream, amber, burnt orange and deep brown. Barlow Condensed is used for display text, Space Grotesk for interface text and IBM Plex Mono for technical labels. The phone layout provides compact navigation and avoids horizontal page overflow. Status messages and validation feedback accompany upload and processing.

Processing indicators describe the analysis pipeline while the request runs. They are not streamed measurements of individual backend stages. Dashboard activity counts are mutable application records and should not be confused with the fixed training, validation and test sample counts.

13 Testing and limitations

13 1 Functional validation

The integration test exercises incorrect and correct login, unique certificate IDs, issuance and download, hash consistency, original watermark recovery, embedding dimensions and normalization, patch extraction, self-similarity and classifier loading. It then verifies the original, transformed and tampered cases, checks difference-map delivery and confirms required revocation reasons and revoked-status precedence.

The recorded development run passed this integration test and the Next.js production build. Browser checks exercised upload and real verification, desktop and phone layouts, navigation, table filters and cancellation of the revocation dialog. These checks support the implemented workflow; they are not an independent statistical assessment of detection robustness.

13 2 Validity of the experiment

The dataset has 36 base certificates, one template and fixed transformation recipes. There are only six held-out base certificates, and multiple pairs share each reference. Pair counts therefore overstate the diversity of independent documents. Coordinate-based edits and shared replacement text may supply artifacts that are easier to classify than realistic forged content.

The split prevents the same base from crossing partitions, but similar names, course patterns, rendering style and transformation rules remain shared. No external test corpus, unfamiliar-template evaluation, repeated grouped cross-validation, probability calibration or systematic adversarial stress test was conducted. No throughput or latency benchmark is reported.

13 3 Technical and security limits

- Pretrained ImageNet features are not semantic certificate text verification; no OCR is implemented.
- Center cropping, fixed resizing and simple rotation can lose or distort detail. Skew, crop and perspective alignment are not corrected.
- The LSB watermark is fragile, unkeyed and reproducible. SHA-256 alone does not authenticate an issuer.
- A trusted registry is required. Compromised or incorrectly registered references are outside the present threat model.
- Public local-file routes, fixed demo credentials and in-memory sessions need production hardening and privacy controls.

Accordingly, the system should be presented as an evaluated course prototype for reference-based visual comparison, with human review required for uncertain or consequential decisions.

14 Conclusion and reproducibility

14 1 Conclusion

Credence AI demonstrates a complete reference-based certificate analysis workflow using real neural inference and a trained supervised classifier. A frozen pretrained ResNet18 supplies global and regional representations, while Random Forest combines their similarities with structural and pixel measurements. The system distinguishes the demonstrated harmless transformation from the controlled grade alteration and integrates registry revocation with the visual result.

The grouped synthetic experiment achieves 100 percent accuracy on 90 held-out pairs, compared with 81.11 percent for global-only features. The evidence supports the usefulness of regional features within the controlled setup. Dataset size, shared layout and predictable edits prevent extrapolation to universal real-world accuracy.

14 2 Future work

The first priority is a larger, independently sourced dataset covering multiple institutions, layouts, scanners and realistic edits. Future evaluation should hold out templates and alteration types, repeat grouped splits and assess confidence calibration. Document registration and OCR-based comparison of name, grade and course fields would address important semantic and alignment weaknesses. Certificate-specific fine-tuning or Siamese networks can then be investigated with sufficient data. Robust watermarking and stronger issuer authentication are separate security improvements.

14 3 Running and reproducing the project

From the submitted project root, create the Python environment and install requirements. Install frontend packages using the retained lockfile. Model and result files are included in the project directory; training can regenerate the classifier and evaluation.

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
cd frontend
npm ci
cd ..

PYTHONPATH=backend .venv/bin/python -m app.ai.train_classifier
./start-backend.sh
```

In a second terminal, run npm run dev from frontend and open http://localhost:3000. The demonstration account is admin@iiitk.ac.in with password credence123. The initial pretrained-weight download and dependency installation require internet access.

```
PYTHONPATH=backend .venv/bin/pytest -q
# From frontend
npm run build
```

Key evidence files are dataset.json and metrics.json under backend/app/results, tamper_classifier.joblib under backend/app/models, and tests/test_system.py. Exact package resolutions are retained in requirements.lock.txt and frontend/package-lock.json.

References and development acknowledgement

[1] K. He, X. Zhang, S. Ren and J. Sun, Deep Residual Learning for Image Recognition, Proceedings of CVPR, pp. 770-778, 2016.

https://arxiv.org/abs/1512.03385

[2] L. Breiman, Random Forests, Machine Learning, vol. 45, pp. 5-32, 2001.

https://www.stat.berkeley.edu/users/breiman/randomforest2001.pdf

[3] Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, Image Quality Assessment From Error Visibility to Structural Similarity, IEEE Transactions on Image Processing, vol. 13, no. 4, pp. 600-612, 2004.

https://ece.uwaterloo.ca/~z70wang/research/ssim/

[4] PyTorch contributors, torchvision ResNet18 model and pretrained-weight documentation. Accessed 7 October 2026.

https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.resnet18.html

[5] Scikit-learn contributors, RandomForestClassifier API documentation. Accessed 7 October 2026.

https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

Implementation evidence

The system description is based on backend/app/main.py, services/vision_service.py, services/pdf_service.py, ai/dataset.py and ai/train_classifier.py. Reported test results come from backend/app/results/metrics.json and the saved evaluation figures. Demonstration measurements come from retained verification records for certificate CRED-IIITK-2026-000003. Interface figures are captured from the implemented application.

Use of AI assisted development tools

AI-assisted coding tools supported implementation, debugging, interface development and documentation. The project reuses openly available pretrained weights and libraries, and trains a separate classifier on generated certificate comparisons. This report identifies the reused model, the task-specific training procedure, the measured results and the limitations without claiming original ResNet training or independent real-world validation.

Submission materials

The report is provided as a PDF for the report submission link and as an editable Word document. The project source, dependency manifests, trained classifier, dataset metadata, evaluation figures and run instructions belong in the separate project archive. No presentation file is required for the described submission workflow.

15 Key code and repository

Browse source code and reports on GitHub

Selected Python excerpts, formatted for readability. Variables are defined in the linked source files; these are not standalone programs.

Pretrained visual feature extractor

backend/app/services/vision_service.py

```
weights = ResNet18_Weights.DEFAULT
model = resnet18(weights=weights)
model.fc = torch.nn.Identity()
model.eval()
```

The final classification layer is removed so ResNet18 returns visual features.

Keep related samples in one split

backend/app/ai/train_classifier.py

```
ids = np.random.default_rng(42).permutation(36)
tr = np.isin(groups, ids[:24])
va = np.isin(groups, ids[24:30])
te = np.isin(groups, ids[30:])
```

The split is by base certificate, not by individual transformed image.

Fit and select the classifier

backend/app/ai/train_classifier.py

```
for name, m in candidates.items():
    m.fit(x[tr], y[tr])
    s = f1_score(y[va], m.predict(x[va]))
    if s > bestscore:
        best, bestscore = (name, m), s
```

Logistic Regression and Random Forest are fitted; validation F1 chooses the winner.

Combine AI with integrity and revocation

backend/app/main.py

```
match = hashlib.sha256(data).hexdigest() == obj['documentHash']
result = ('TAMPERING_SUSPECTED' if pred else
          ('VERIFIED' if match else 'TRANSFORMED_COPY'))
if confidence < .65: result = 'REVIEW_RECOMMENDED'
if obj['status'] == 'REVOKED': result = 'REVOKED'
```

Prediction supplies the class; file hash, confidence and registry status determine the result.